

Computer Vision (CAP4410) Assignment 2

Lukas Kelk

Computer Engineer

Department of Computer Science
Florida Polytechnic University, Florida, USA

February 16, 2025

CONTENTS

CONTENTS	ii
LIST OF FIGURES	iii
1 INTRODUCTION	1
2 IMPLEMENTATION	2
2.0.1 Explanation Of Tkinter GUI to Usable CV2 Images	2
2.0.2 Explanation Of Tkinter GUI	2
2.0.3 Explanation Of Otsu Binary Thresholding	5
2.0.4 Explanation Of Otsu Multi-Thresholding	7
2.0.5 Explanation Of Mean Shift Segmentation	8
2.0.6 Discussion	9
REFERENCES	11

LIST OF FIGURES

Figure 2.1	Full Tkinter GUI	3
Figure 2.2	Segmentation Dropdown Box	3
Figure 2.3	Image Dropdown	3
Figure 2.4	Otsu Method (2 Classes) Image 1	5
Figure 2.5	Otsu Method (2 Classes) Image 2	6
Figure 2.6	Otsu Method (2 Classes) Image 3	6
Figure 2.7	Otsu Method (Three Classes) Image 1	8
Figure 2.8	Otsu Method (Three Classes) Image 2	8
Figure 2.9	Otsu Method (Three Classes) Image 3	8
Figure 2.10	Mean Shift Image 1	9
Figure 2.11	Mean Shift Image 2	9
Figure 2.12	Mean Shift Image 3	9

1 INTRODUCTION

I have applied three image segmentation techniques in this Assignment: two-class Otsu's binarization, multi-thresholded Otsu's method, and the Mean Shift segmentation. I applied the method of Otsu to segment images into distinct foreground and background regions by computing optimal threshold values first for two classes and later generalized to multiple classes for segmentation of a higher level. The process of Mean Shift was applied to detect groups of pixels in the feature space to enhance segmentation by aggregating pixels with comparable features. A graphical user interface (GUI) was also designed to help users see and interact with the results of segmentation in real-time. This report discusses the implementation, results, and conclusions obtained with the help of these segmentation techniques.

2 IMPLEMENTATION

2.0.1 Explanation Of Tkinter GUI to Usable CV2 Images

In order to calculate any type of filter or computation on an image and display it using Tkinter we need to convert the CV2 read image into a Tkinter display image. When CV2 reads an image it uses a numPY array to store the values. In order for Tkinter to be able to read and display the image we need to convert its color space from Blue Green Red to Red Green Blue and convert its data type from numPY array to Tkinter photoImage. To facilitate the conversion from numPY array to Tkinter Photoimage we can use the Pillow library, this acts as the intermediate between them.

```
1      #convert OpenCV image (BGR) to (RGB) format
2      stackConvert1 = cv2.cvtColor(stack1, cv2.COLOR_BGR2RGB)
3      stackConvert2 = cv2.cvtColor(stack2, cv2.COLOR_BGR2RGB)
4      #convert the initial image to a format for Tkinter after the root window is
       ↳ created
5      photo1 = ImageTk.PhotoImage(Image.fromarray(stackConvert1))
6      photo2 = ImageTk.PhotoImage(Image.fromarray(stackConvert2))
7      #Tkinter display and pac
8      imageTop = tk.Label(root, image=photo1)
9      imageTop.pack(padx=10, pady=10)
```

And the other way as well

```
1      #convert TKimage to a PIL Image
2      image = ImageTk.getimage(image)
3      #convert to numpy array
4      imageNP = np.array(image)
5      #convert color
6      imageNP = cv2.cvtColor(imageNP, cv2.COLOR_RGB2BGR)
```

2.0.2 Explanation Of Tkinter GUI

For this assignment Tkinter GUI was the best option to be used. It allows all the images and options to be in one window instead of separate ones. Below is the full screenshot of the Tkinter GUI made

for this Assignment, the bottom left features two drop boxes. The left one allows you to select what type of Image Segmentation you want to select, between Otsu Method many and 2 classes as well as Mean Shift Segmentation. the right dropdown allows you to select between the photos present in the program - 3. The apply button will apply the Segmentation to the bottom image changing it to reflect the segmentation you selected. The close button will quit out of the app and the reset button will reset the bottom image to the original image. Tkinter only had a slight learning curve compared to using npStack, but it turned out to be a better option because of its look and ease of use with the options we needed.

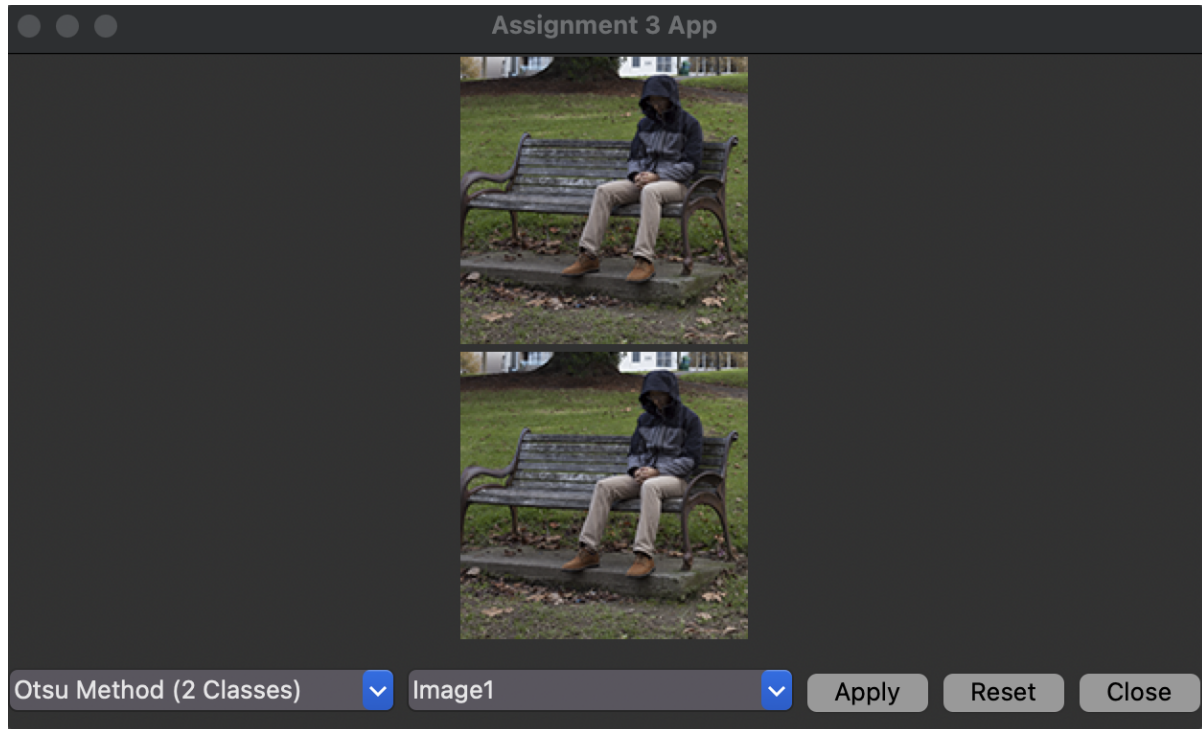


Figure 2.1. Full Tkinter GUI

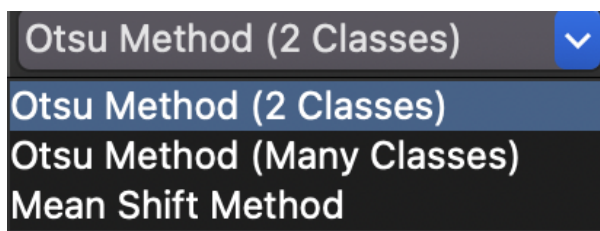


Figure 2.2. Segmentation Dropdown Box

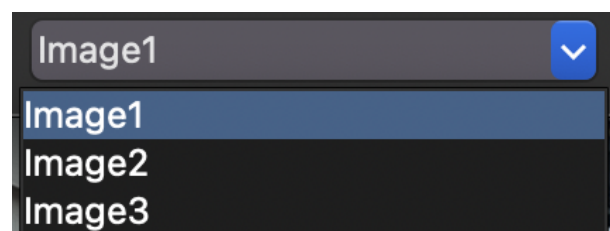


Figure 2.3. Image Dropdown

```

1  #create tkinter window
2  root = tk.Tk()
3  root.title("Assignment 3 App")
4
5  imageT = imageB = cv2.imread('Assignment3/Image1.png')
6  photo1 = ConvertCV2ToTKINTER(imageT)
7  photo2 = ConvertCV2ToTKINTER(imageB)
8
9  #label to display image
10 imageTop = tk.Label(root, image=photo1)
11 imageTop.pack(padx=10)
12 imageBottom = tk.Label(root, image=photo2)
13 imageBottom.pack(padx=10)
14
15 #keeps image refernec so scope dosent destroy it
16 imageBottom.image = photo2
17 imageTop.image = photo1
18
19 #create a dropdown menu for filter selection
20 filterCombo = ttk.Combobox(root, values=[
21     "Otsu Method (2 Classes)",
22     "Otsu Method (Many Classes)",
23     "Mean Shift Method"
24 ], state="readonly")
25
26 filterCombo.set("Otsu Method (2 Classes)")
27 filterCombo.pack(side=tk.LEFT, pady=10)
28 #bind filter selection event
29 filterCombo.bind("<<ComboboxSelected>>", lambda event: updateGUI())
30
31 #dropdown for number of classes (Initially hidden)
32 classCombo = ttk.Combobox(root, values=[str(i) for i in range(3, 11)],
33     ↪ state="readonly")
34 classCombo.set("3") # Default to 3 classes
35
36 #dropdown for filter dimension
37 combo = ttk.Combobox(root, values=['Image1', 'Image2', 'Image3'],
38     ↪ state="readonly")
39 combo.set('Image1')
40
41 combo.pack(side = tk.LEFT, pady=10)
42
43 combo.bind("<<ComboboxSelected>>", lambda event: changePIC())
44
45 #Buttons
46 applyButton = tk.Button(root, text="Apply", command=applyFilter)
47 applyButton.pack(side=tk.LEFT, pady=10)
48 resetButton = tk.Button(root, text="Reset", command=changePIC)

```

```

46 resetButton.pack(side = tk.LEFT, pady=10)
47 closeButton = tk.Button(root, text="Close", command=root.quit)
48 closeButton.pack(side = tk.LEFT, pady=10)
49
50 #run the Tkinter window
51 root.mainloop()

```

2.0.3 Explanation Of Otsu Binary Thresholding

As we have binary images, the mission of the Otsu's method is to attempt to seek a value of t that will cause the weighted within-class variance expressed by the formula:

$$\sigma_w^2(t) = q_1(t)\sigma_1^2(t) + q_2(t)\sigma_2^2(t) \quad (2.1)$$

$$q_1(t) = \sum_{i=1}^t P(i) \quad \& \quad q_2(t) = \sum_{i=t+1}^I P(i) \quad (2.2)$$

$$\mu_1(t) = \sum_{i=1}^t \frac{iP(i)}{q_1(t)} \quad \& \quad \mu_2(t) = \sum_{i=t+1}^I \frac{iP(i)}{q_2(t)} \quad (2.3)$$

$$\sigma_1^2(t) = \sum_{i=1}^t [i - \mu_1(t)]^2 \frac{P(i)}{q_1(t)} \quad \& \quad \sigma_2^2(t) = \sum_{i=t+1}^I [i - \mu_2(t)]^2 \frac{P(i)}{q_2(t)} \quad (2.4)$$

(See Reference 2) Otsu Binary (2 class) method will seek out a threshold on the histogram that best separates the image based on the variance of the frequency of all the pixels. This threshold will then be used as a cut off point. For example if $t = 170$, any pixel values above 170 will be white and any below 170 will be black, this can be useful to segment specific types of images. For example image three it works well because the image is less complex and has distinct regions.



Figure 2.4. Otsu Method (2 Classes) Image 1

Implementation:



Figure 2.5. Otsu Method (2 Classes) Image 2

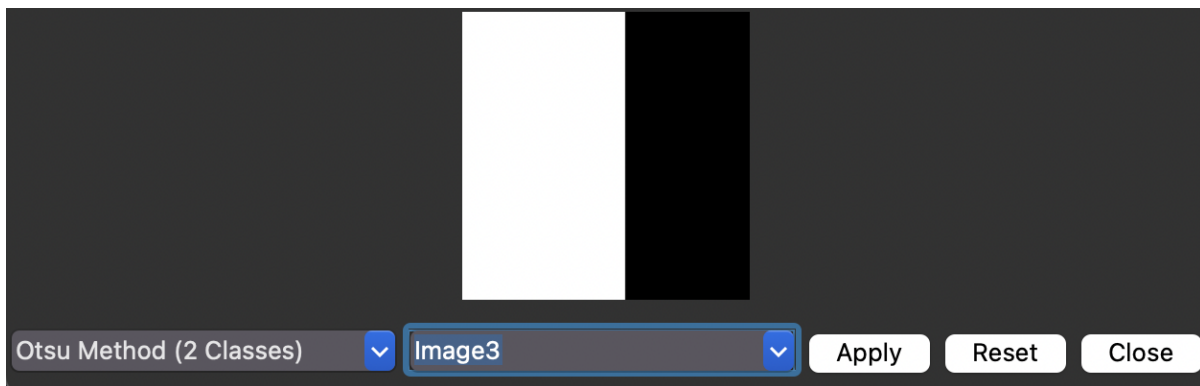


Figure 2.6. Otsu Method (2 Classes) Image 3

```

1  def otsu2Classes(image):
2
3      imageNP = ConvertTKINTERtoCV2(image)
4      gray = cv2.cvtColor(imageNP, cv2.COLOR_BGR2GRAY)
5
6      #this function automatically applies and calculates the threshold using cv2
       ↪ methods
7      #the threshold variable stores the actual value of the threshold
8      threshold, filteredImage = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
       ↪ cv2.THRESH_OTSU)
9
10     #convert back to CV2
11     filteredImage = ConvertCV2ToTKINTER(filteredImage)
12
13     #update display and image
14     imageBottom.config(image=filteredImage)
15     imageBottom.image = filteredImage
16
17     return

```

2.0.4 Explanation Of Otsu Multi-Thresholding

Otsu Multi-Thresholding is a similar concept to Binary thresholding but instead of picking one threshold it picks multiple thresholds which can be user defined, these thresholds are calculated on the same concept as the one threshold, but instead its separated into a defined amount of classes. Also we used numpys built in color map to map colors to each of the regions.

```
1 def otsuManyClasses(image):
2     imageNP = ConvertTKINTERtoCV2(image)
3     #make image gray
4     gray = cv2.cvtColor(imageNP, cv2.COLOR_BGR2GRAY)
5
6     #we can change the class count here
7     classes = int(classCombo.get())
8
9     #qpply Multi-Otsu Thresholding
10    thresholds = threshold_multiotsu(gray, classes=classes)
11
12    #assign labels to different regions
13    regions = np.digitize(gray, bins=thresholds)
14
15    #convert regions fomr 64 bit to 8 bit for tkinter
16    regions = util.img_as_ubyte(regions)
17
18    #generate a colormap based on matplots preset tab10 colorset, so the colors
19    ↪ are always distincly different
20    #than normalize those color values to scale to grey scale
21    colormap = np.array([plt.cm.get_cmap("tab10")(i)[:3] for i in
22    ↪ range(classes)]] * 255
23
24    #empty image to cast colors to
25    color = np.zeros((*gray.shape, 3), dtype=np.uint8)
26
27    #apply colors to each of the regions in the range
28    for i in range(classes):
29        color[regions == i] = colormap[i]
30
31    #convert back to Tkinter format
32    filteredImage = ConvertCV2ToTKINTER(color)
33
34    #update the Tkinter image label
35    imageBottom.config(image=filteredImage)
36
37    #keep image reference
38    imageBottom.image = filteredImage
39
40    return
```



Figure 2.7. Otsu Method (Three Classes) Image 1

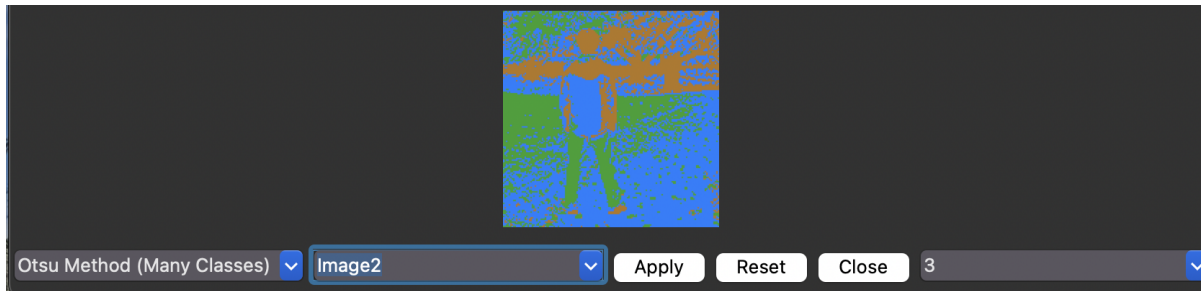


Figure 2.8. Otsu Method (Three Classes) Image 2

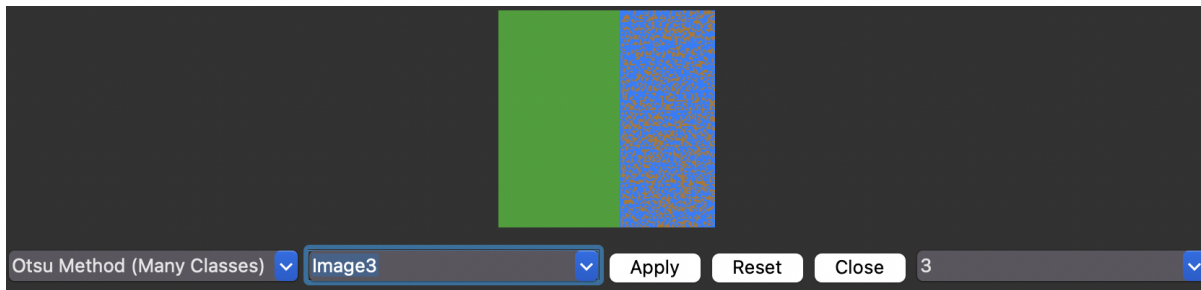


Figure 2.9. Otsu Method (Three Classes) Image 3

2.0.5 Explanation Of Mean Shift Segmentation

Mean Shift is a clustering-based image segmentation technique that iteratively shifts data points toward the densest regions in the feature space. The steps are as follows:

1. Initialize each point x as a candidate for convergence.
2. Compute the mean shift vector using the kernel function.
3. Update the position by shifting the point toward the new mean.
4. Repeat until convergence (shifts become negligible).

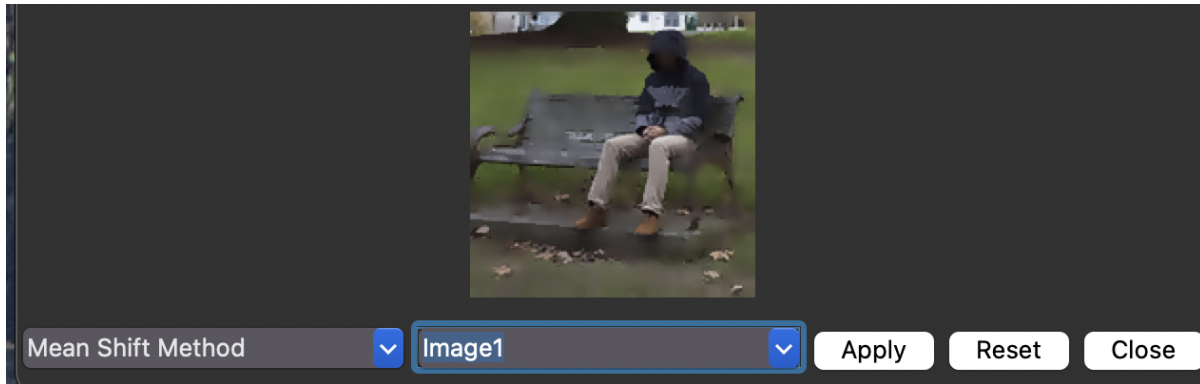


Figure 2.10. Mean Shift Image 1

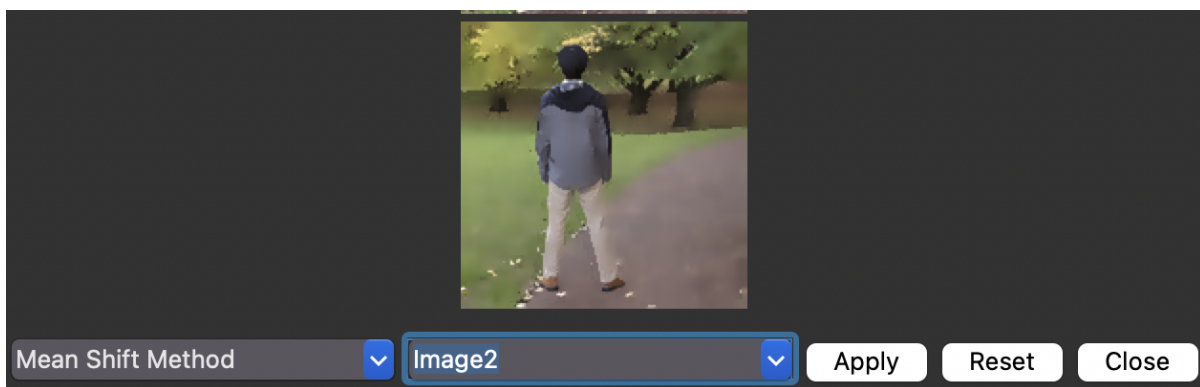


Figure 2.11. Mean Shift Image 2

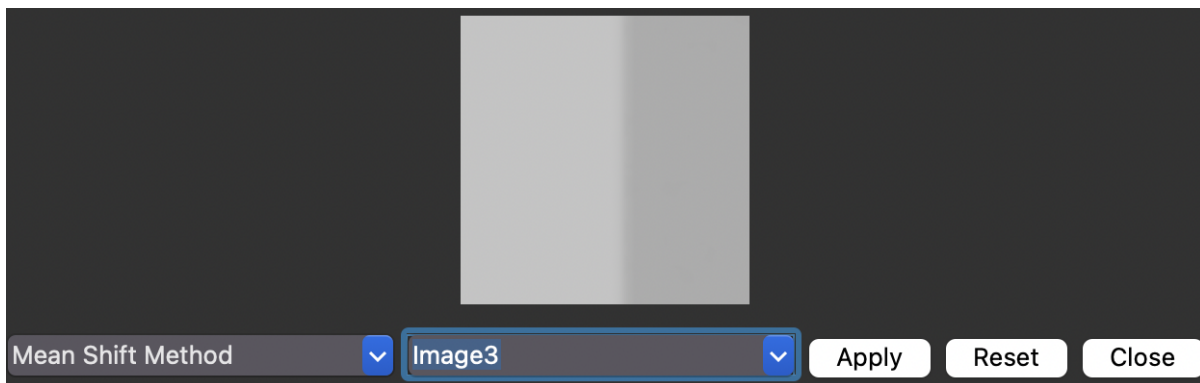


Figure 2.12. Mean Shift Image 3

2.0.6 Discussion

The implementation of this assignment explores three different image segmentation approaches: the two-class model using Otsu's binarization, Otsu's method with multiple thresholds, and Mean Shift segmentation. Additionally, a graphical user interface (GUI) was developed using Tkinter to facilitate interactive experimentation with these segmentation techniques. The **Otsu's Method for Two Classes** function converts an image to grayscale and applies Otsu's thresholding method

using OpenCV. This technique automatically determines an optimal threshold value to separate the image into foreground and background. The segmented result is then converted into a format suitable for display in Tkinter. Expanding on this, the **Otsu's Method for Multiple Classes** extends the thresholding process beyond a simple binary partition, allowing segmentation into multiple classes. This is implemented using the `threshold_multiotsu` function from the `skimage.filters` module. The GUI provides a dropdown menu for selecting the number of classes, and the segmented regions are assigned distinct colors using a Matplotlib colormap to enhance visual differentiation. The **Mean Shift Segmentation** method was implemented using OpenCV's `pyrMeanShiftFiltering` function. This technique clusters pixels based on their spatial and chromatic similarities, controlled by the spatial radius (sp) and color range radius (sr) parameters. These parameters were carefully optimized to achieve meaningful segmentation while preserving important details in the image. Overall, this assignment successfully demonstrates various image segmentation techniques and their effectiveness in distinguishing different regions within images. The interactive GUI enhances usability, enabling users to experiment with different segmentation parameters and immediately visualize their impact.

REFERENCES

- GeeksforGeeks (2025a). "Image segmentation using mean shift clustering. Accessed: 2025-03-01.
- GeeksforGeeks (2025b). "Python — thresholding techniques using opencv (set 3: Otsu thresholding). Accessed: 2025-03-01.
- LearnOpenCV (2025). "Otsu thresholding with opencv. Accessed: 2025-03-01.
- Madrigal, R. (2025). "Image segmentation using thresholding and otsu's method. Accessed: 2025-03-01.
- Matplotlib (2025a). "Colormap reference. Accessed: 2025-03-01.
- Matplotlib (2025b). "Matplotlib tutorial: How to use colormaps. Accessed: 2025-03-01.
- of Bonn, U. (2025). "Segmentation lecture slides. Accessed: 2025-03-01.
- OpenCV (2025). "Image Thresholding in OpenCV. Accessed: 2025-03-01.
- OpenCV (2025a). "Mean shift and camshift. Accessed: 2025-03-01.
- OpenCV (2025b). "Mean shift and camshift (opencv 3.4). Accessed: 2025-03-01.
- OpenCV (2025c). "Opencv github repository. Accessed: 2025-03-01.
- Overflow, S. (2025). "Applying multi-otsu threshold for my image. Accessed: 2025-03-04.
- PyImageSearch (2025). "Opencv thresholding: cv2.threshold(). Accessed: 2025-03-01.
- Scikit-Image (2025). "Multi-otsu thresholding example. Accessed: 2025-03-01.
- TutorialsPoint (2025). "Combobox Widget in Python Tkinter. Accessed: 2025-03-01.
- with Python, C. V. (2025). "Otsu thresholding - full tutorial (opencv). Accessed: 2025-03-01.